

Optimisations using Java Vector API

Speeding up Lucene vector similarity through the Java Vector API

In Figure 8, 'dim' is the dimension/length of the float[] arrays in test and 'score' is the number of dot product/Euclidean distance operations done per second. 'Gain' is the increase in speed of the vectorized implementation against the corresponding default implementation.

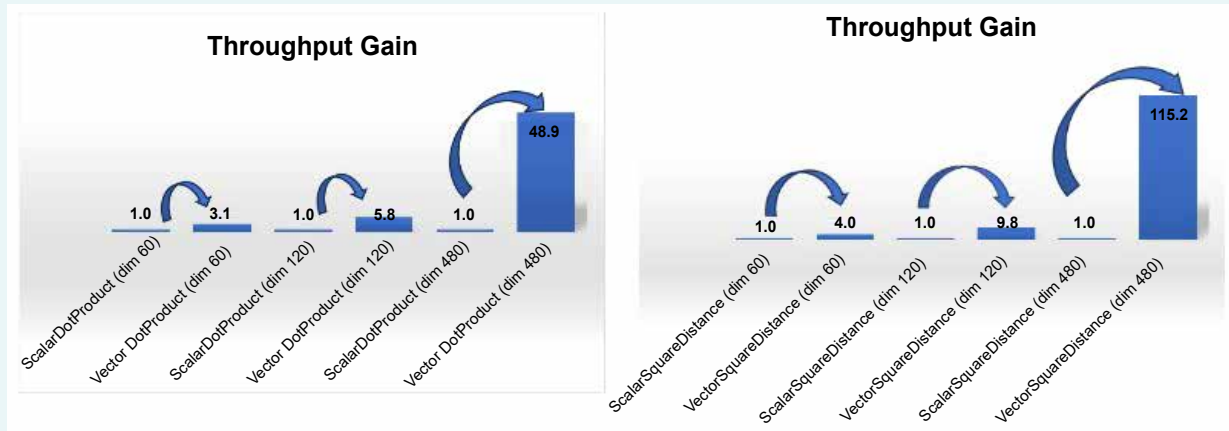


Figure 8: Lucene vector similarity through the Java Vector API

- Lucene is a search engine library used in applications like Elasticsearch, OpenSearch, and Solr.
- Approximate nearest neighbour (ANN) search was introduced in Lucene 9.
- Calculating distances (dot-product, Euclidean distance, and cosine) between two vectors in vector space is a compute-intensive task.
- Lucene's implementation of ANN relies on a scalar implementation of the vector similarity functions 'dot-product', 'Euclidean distance', and 'cosine'.
- Distance calculation can be increased multi-fold by vectorizing the scalar implementation. As the dimensions increase, the performance gains from vectorization increase, improving by ~50 times and ~115 times for dot product and square distance computations, respectively. With vector dimensions of 300 or higher, the efficiency gains are even more significant (do refer to Lucene JIRA #12091).

JavaFastPFOR optimisation

- JavaFastPFOR is a library designed for the efficient compression and decompression of integers.
- This library is well-regarded and used within Apache Pinot, a real-time distributed OLAP datastore, and is already incorporated into the file format, Parquet.
- The VectorJavaFastPFOR (re-written with Java Vector API) algorithm compresses integers in blocks of 256 integers and provides a promising performance benefit ranging from around 4x to 10x improvements for vectorized implementations.
- There is also a proposal (Lucene JIRA #12090) for integration of this library within Lucene's posting file format.

Graceful degradation

Taking into consideration that everything has its limitations, the Java Vector API aims to degrade gracefully. What does this mean?

Although the aim is to make vector calculation platform-independent, sometimes, the high-level language vector instructions may not be compiled by the JIT compiler to optimise vector instructions they are supposed to. This could be because they are not supported on

that hardware, or because of some limitation of the JIT compiler to emit vectorized instructions in certain specific cases. In such cases the Vector API implementation should degrade gracefully and still work. This means

that the result produced should still be the same as in the case of non-vector scalar implementation (like we do without vectors using, say, the 'for' loop). This is termed graceful degradation in Java Vector API. **END** 🐧

This article is based on a joint speaking session by **Sourav Kumar Paul**, engineering manager at Intel, and **Abhijit Kulkarni**, senior software engineer, Intel, at Open Source India 2023.

Compiled by Shivangi Kharoo, tech journalist.